

奇普计划白皮书与技术规格

一句话定位：奇普档案是”文明的自解压文件”——打开它的人还没有出生，所需的软件一个都不剩，但没关系：解读方法和解码程序就写在文件自己身上。

版本 0.2.0 / 2026 读者：机构决策者（第一、二、四部分）与实现工程师/AI 编程助手（第三部分）

第一部分意义与方案

1. 摘要

奇普计划 (Quipu Project) 定义一种**自描述归档容器格式**及配套工具链。一个奇普档案 (.quipu) 在被打开时，不假设读者拥有任何当代操作系统、指令集、编解码软件或文档——它只假设读者能读字节和 ASCII 英文 (或者，是一个能读英文的大语言模型)。档案开头的一小段明文，教会读者执行一台四规则的通用图重写机；这台机器再执行档案内携带的编解码程序，从而解锁全部内容。档案同时内嵌测试对，使任何读者都能**机械地验证**自己的解码是否正确，而不必信任任何外部权威。

我们把”格式会过时”这个保存领域的根本难题，转化为一次性的、不足 2.5KB 的教学成本。数据不需要随格式迁移而反复转码；语义的钥匙与数据封存在同一个文件里。

2. 问题：我们建造在流沙上的数字文明

当代信息载体的结构性脆弱：

- **介质层**：磁盘与磁带寿命 5–10 年，需恒温恒湿与持续维护；
- **格式层**：编码格式平均 10–20 年过时。今天的 JPEG、H.264、docx，都依赖特定的、复杂的、终将消失的软件栈；
- **生态层**：x86、ARM 这样的指令集，Linux、Windows 这样的 ABI，都是历史的偶然产物。它们消亡后，为它们编译的程序全部成为死字节。

现行的数字保存策略是**永续迁移**：每隔若干年，把全量数据转码到新格式。对 PB 级机构而言，每次迁移意味着数月算力、人工校验与不可逆的损坏风险，且成本**每十年重新支付一次，直到永远**。迁移链条任何一环中断——一场战争、一次预算削减、一个机构的解散——数据就在”比特还在”的意义上活着，在”无人能读”的意义上死去。

更根本的问题是：我们今天写入的一切，都默认了读者与我们是同时代的人。

3. 文明的断代：结绳记事的教训——本计划名字的由来

3.1 奇普：一把丢失的钥匙如何抹掉一个文明

16 世纪的安第斯，印加帝国用**奇普 (Quipu)**——打了结的绳串——记录人口、税赋、仓储乃至叙事。西班牙殖民者销毁了绝大多数奇普，处死了读绳的专职记录者。绳结留了下来，读法死了。

此后四百年，学界的主流判断是：印加没有文字，奇普只是”辅助记忆的珠子”。直到近二十年，研究者才逐渐证实奇普至少有数字的十进制记法，且很可能编码了远超数字的信息——一个文明的**记录系统**一直躺在博物馆里，只是被误判为”原始人的记事绳”。而读懂它的全部代价，本来不过是一本”结绳规则手册”。

玛雅象形文字是同一教训的另一半：碑文遍布丛林，读法随抄写阶层消亡而失传，被当作”神秘图画”三百年。破译之所以可能，靠的是两样幸存物：一把”钥匙”(传教士兰达记录的不完全字母表) 和大量**已知答案的样本** (历法数字)——研究者先验证历法读对了，才敢信自己读懂了别的。

这两个案例给出数字保存的判词与配方：

文明的断代，往往不是数据没了，而是”数据如何被读”没了。没有钥匙的数据，会被后代误认为没有数据。 钥匙 + 已知答案的样本 = 重新破译的可能。

奇普计划的全部设计，就是把那次侥幸成功的破译，变成每一次封存时的必然：把钥匙刻在数据自己身上。我们以那个最著名的失败命名，是为了让每个使用者都记得这个计划为什么存在。

3.2 时间分层：从一万年前回到今天

保存方案必须与”假设多少东西还活着”匹配。奇普计划按时间尺度分四层，每层只依赖比它更长的那一层假设：

时间尺度	假设还活着的	钥匙的形式	正文的形式
一万年 +	只有视觉与直觉	图形规则卡：物理雕刻的三个符号（擦除 / 复制 / 组合）与三条图示重写规则，用连续三帧图解一次完整归约——任何有眼睛的读者都能”看图学会”图计算	石刻/陶瓷上的图项与图像，语义即图示
一千年	数学与某种语言	自然语言写就的规格文档（数学记号 + 多语言互译冗余）	图项 + 源码级描述
一百年	LLM 或工程师、ASCII 英文	522 字节 ASCII 明文头：教会四规则图重写机	SKI 图项 + QuipuVM 字节码 + 负载（本规格主体）
今天	x86、现有软件生态	不需要钥匙	i386 机器码附赠轨、普通文件原样封装

两个结构性性质：

- 1. 嵌套兜底：一百年层失败，退化到一千年层；再失败，退化到一万年层。每一层都是上一层的保险而非替代。完整存档是同一份语义的四重誊写，誊写的语言一层比一层原始。
- 2. 钥匙随数据衰减：时间越长，钥匙越”不依赖文化”——从英文退到数学，从数学退到图形直觉。而正文始终可以是同一套图重写语义：图计算是唯一在全部四个时间尺度上都能充当正文的形式——一万年前的图形规则卡 and 一百年后的SKI 头不是两个方案，是同一根语义主轴在时间轴上的投影。

4. 前提：三样赌注

奇普档案的生存只依赖三条假设，按脆弱程度排序：

- 1. 读者能读字节——任何计算文明都满足；
- 2. ASCII 英文可读——人类存续则自然语言存续；大语言模型存续则英文技术文本必然是其语料的一部分。这是所有可选通信协议中历史表现最好的一个（ASCII 已存活 60 余年,UTF-8 是它的超集）；
- 3. 存在能”读规格、写程序、跑测试”的执行者——人类工程师或大语言模型均可，不挑。

凡不在此三条内的知识——任何指令集、任何文件格式、任何软件——都不允许出现在档案的关键路径上。我们特别区分两类知识：

	自我再生的知识	只会衰减的知识
例子	算术、组合子逻辑、通用计算	x86 指令集、JPEG 算法、PNG 格式
被遗忘后	任何计算文明都会重新发明	永远不会被重新发明
档案策略	只需”指名唤醒”	必须完整携带（作为程序或规格）

档案的正文只能用不会死的语言书写；会死的知识，要么被教会，要么只当附赠。

5. 核心理念

5.1 存档即教学

奇普档案本质上是写给未来的一封**可执行教材**。教学成本占大头且不可压缩——在一个只存一幅 32×32 圆盘的演示档案里，教学文本占 83%。但这笔成本**只付一次**，之后任意大的负载都沿用同一套已教会的语义，边际成本趋近于零。

5.2 自然语言只出现在第一层

自然语言无法机械验证。因此设计纪律是：**任何能从自然语言降格为程序的知识，必须降格**。明文头（2.5KB）只负责教会一台四规则的机器；此后的一切——编解码器、格式规格、索引——全部是形式化的程序与数据。误读风险被压缩在最小范围，且由内嵌测试对兜底。

5.3 验证即信任

未来读者不该信任我们。档案内嵌三层测试对：

- **语言层**：三个图项必须归约到 41、7、5；
- **指令层**：26+ 条测试向量，锁定虚拟机每条指令的正常路径与全部错误路径；
- **端到端**：每段媒体负载附”解码结果的已知前缀与哈希”。

批改权随档案封存，不依赖任何活着的权威机构。这也是对 AI 读者”幻觉”问题的根本回答：档案不要求 AI 自信，只要求 AI 通过考试。

5.4 正确性是档案的本职，效率是读者的本职

档案内的编解码器以简洁、无歧义为先，不追求执行速度。未来读者在正确解码一次之后，有无穷的办法加速：JIT、重写、用它的时代的原生工具重建。档案只需保证”第一次一定对”。

6. 采纳经济学：为什么今天的人愿意打包

6.1 核心账：把”保险”卖给怕”迁移”的人

现状： 每 5-10 年，格式过时/介质老化 → 全量转码一次
1PB 数据，每次迁移 = 数月算力 + 人工校验 + 出错风险
成本每 10 年重复一次，直到永远
奇普： 一次性打包（+1~3% 体积开销）→ 之后零迁移

6.2 今天就有用

- **当下可用的容器**：原样封装区一键还原为普通文件，用现有软件直接打开；原生编码区配 WASM 查看器，双击即读。用户的数据从不被”锁进”陌生格式；
- **零迁移保险**：替代”每十年一次”的全量转码项目；
- **数据溯源凭证**：自描述 + 内置 CRC + 自带解码器的容器，天然满足 AI 训练语料、科研数据的来源与完整性证明需求——当下即可变现。

6.3 目标客户

不面向消费者。面向**持有法定或使命性长期保存义务的机构**：国家与大学图书馆、档案馆（可嵌入 WARC/BagIt 工作流）、科研数据仓（天文、基因组、粒子物理）、合规存档（金融、医疗、航空）。

6.4 成本对比

	现状 (永续迁移)	奇普
每 10 年	全量转码: 数月算力 + 人工 + 损坏风险	零
一次性成本	—	打包 <1% 体积 + A 级选择性转码
失败模式	迁移链断裂 → 静默死亡	规格、程序、测试对与数据同生死

第二部分架构

7. 总体结构

第 1 层	ASCII 明文头 (≤2.5KB) 教会:SKI 图重写机 + 容器格式 + 测试对惯例	唯一一次教学成本
第 2 层	程序节区 • QuipuVM 解释器 (SKI 图项,~500B) • 编解码器 (QuipuVM 字节码) • 测试向量 (26+ 条)	权威语义 格式能力 信任锚
第 3 层	数据节区 • A 级: 原生编码的核心内容 • B 级: 原样封装 + 规格随葬 • C 级: 冷数据, 仅登记	99% 的体积
MANIFEST(ASCII 清单)		

8. 第一层:SKI 图重写机

语义底座是组合子逻辑 (Schönfinkel 1924 / Curry) 的四原子变体:S、K、I 组合子加整数运算。完整语义只有七条重写规则, 一个够格的读者用约 60 行代码即可重建求值器。选择它的原因不是效率, 而是**可教学性**: 它是人类已知的最小通用计算形式之一, 且属于”自我再生的知识”。

9. 第二层:QuipuVM——为”被解释”而生的虚拟机

若每种编解码器都直接写成 SKI 图项, 体积将膨胀 10-100 倍且无法维护。QuipuVM 是一台定长指令寄存器虚拟机 (16×32 位寄存器、22 条指令、每条指令固定 8 字节、无标志寄存器、无未定义行为、全机器仅 6 种错误码)。它的解释器用 SKI 图项写就 (约 300-500 字节), 随档封存——这份程序本身就是 **QuipuVM 的权威语义, 形式化、可执行、可验证, 不依赖一行自然语言**。

设计取向刻意反主流: 定长指令浪费 2-3 倍空间, 寄存器机不如栈机紧凑——全部接受。QuipuVM 的优化目标不是省字节, 而是**不可误读**: 每条指令语义互不依赖, 可乱序教学、乱序验证; 任何两个独立重建的解释器应当在全部 26 条测试向量上逐字节一致。

今天, 存量解码器 (RLE、DPCM, 乃至 zlib 的 inflate) 由编译器从 C 子集自动编译为 QuipuVM 字节码; 一百年后, 读者重建 SKI 求值器、执行解释器、运行字节码——三层解释叠加很慢, 但第一次解码一定是**对的**。

10. 第三层：数据分级

级	内容	处理	体积占比
A	文明重启核心：教科书、技术 图纸、格式规格文档	全量转码为档案原生格式	小
B	重要存量：影像、科研数据、文 献	原样封装（字节不动）+ 格式 规格随葬 + 采样测试对	大
C	海量冷数据	原样封装，仅登记清单	最大

分级的经济学：格式的复杂度只付一次（每格式随葬一份规格,50KB–1MB），数据的规模一分钱不付（PB 级数据零转码）。对存量文件的打包是 O(文件大小) 的纯拷贝操作。解码 B 级数据的负担转移给未来——未来的算力便宜，今天的转码算力贵；我们只需保证可解性（规格 + 测试对），不必预付”已解”的成本。

11. 附赠轨：给当代人的便签

档案允许附加当代可直接使用的产物（如 freestanding i386 机器码解码器），但永远标注 IGNORE if you do not know i386; the term-language version is authoritative。它是给今天读者的便利，从不在未来读者的关键路径上。古书可以夹一张给当代人的便签，但正文必须用不会死的语言写。

12. 信任模型

- 1. 数学底座:SKI 语义是数学对象，不随机构、公司、文明兴衰而改变；
- 2. 冗余互证: 每个解码能力有”自然语言摘要 + 形式化程序 + 测试向量”三种独立陈述；
- 3. 自校验闭环:CRC32 护字节，测试对护语义，端到端哈希护意图；
- 4. 去权威化: 验证不需要联网、CA 或任何活着的参与者——这是与区块链式存证的根本区别。

第三部分技术实现规格

本部分为规范性内容：按其实现，无需任何额外上下文即可独立完成打包器、解释器与验证器。实现完成的判定标准是第 22 章验收测试全部通过。

13. 文档约定

- 所有多字节整数均为小端序；所有”32 位整数”均指二进制补码，运算溢出按模 2^{32} 回绕。
- 字节串写作十六进制序列，如 01 13 00 04 00 00 00。
- 所有格式定义为规范性（必须严格实现）；任何”看起来更合理”的偏离都不允许。
- 实现与本文档冲突时，以测试对的期望输出为准，并回报文档缺陷。

14. 容器结构

```
[ASCII 头，以只含 "END" 的一行结束]
重复若干节区：
    SEC id=<n> type=<t> codec=<c> len=<bytes> crc32=<decimal>\n
    <len 字节二进制负载 >
MANIFEST
<ASCII 索引行 >
END-ARCHIVE
```

解析规则: 从头读取, 找到第一个 END\n 行; 之后按节区循环, 遇 MANIFEST 行进入清单, 直到 END-ARCHIVE 行结束。

硬性规则: 二进制负载不允许出现在任何 SEC 头之前; 节区之间无填充、无对齐; 所有 ASCII 行以 \n(0x0A) 结尾, 解析时遇到 \r 直接报错;CRC32 一律 IEEE(zlib 默认), 十进制打印。

15. ASCII 头规格 (固定模板)

模板逐字使用 (仅 <...> 占位符可填实际值), 总长目标 2500 字节:

```
SELF-DESCRIBING ARCHIVE v2 (Quipu). Payload follows line END.
This file explains itself. Everything after END is binary sections,
each announced by an ASCII line. CRC32 below is IEEE 802.3 (zlib crc32),
printed as decimal.

LAYER A - TERM LANGUAGE (SKI graph machine)
Term encoding (prefix, depth-first, binary):
  00 = integer literal, next 4 bytes little-endian
  01 = application node, two subtrees follow
  02 = duplication node, one subtree follows (shared; see rule DUP)
  10 = atom S | 11 = atom K | 12 = atom I | 13 = atom INC
  14 = atom ADD | 15 = atom MUL | 16 = atom LT
Reduction (leftmost-outermost spine, repeat to normal form):
  ((K x) y) = x ; (I x) = x ; (((S f) g) x) = ((f x) (g x))
  (INC n) = n+1 ; (ADD a b) = a+b ; (MUL a b) = a*b
  (LT a b) = 1 if a<b else 0 (32-bit two's complement, wrap on overflow)
  DUP node: evaluate the shared subtree at most once, then share result.
VERIFY (decode+reduce; these are ground truth):
  ex1 <len1> bytes -> 41 | ex2 <len2> bytes -> 7 | ex3 <len3> bytes -> 5

LAYER B - QUIPUVM (bytecode machine; decoders are shipped as bytecode)
Full spec: 8-byte fixed instructions, 16 x 32-bit registers.
Authoritative semantics = the term-language interpreter program in
section <interp_sec>. Any prose summary in this header is NOT
normative; if unsure, execute the interpreter.

SECTIONS: each begins with one ASCII line:
  SEC id=<n> type=<t> codec=<c> len=<bytes> crc32=<decimal>
payload follows immediately. Types: spec | program | data | test | font.
Codecs: none | rle | dpcm | quipuvvm | ski | raw.
Archive ends with line MANIFEST, index lines, then END-ARCHIVE.
CRC32(all bytes after END up to MANIFEST) = <crc>
END
```

16. 第 1 层:SKI 图项语言

16.1 编码 (前缀深度优先)

首字节	含义	后继字节
0x00	整数字面量	4 字节小端 (按补码解释)
0x01	应用节点 (APP)	左子树, 然后右子树
0x02	复制节点 (DUP)	1 个子树

Table 5 – continued

首字节	含义	后继字节
0x10–0x16	原子 S,K,I,INC,ADD,MUL,LT	无
0xFF	共享引用	1 字节序号 n(可选实现, 见 16.4)

其他字节为非法, 解码器必须报错, 不允许”跳过”。

16.2 求值语义

项 (f x) 表示 APP(f, x)。反复应用”最左最外”归约, 直到无规则可用:

R-K:	((K x) y)	→ x	
R-I:	(I x)	→ x	
R-S:	((S f) g) x)	→ ((f x) (g x))	[x 以 DUP 节点共享, 而非复制]
R-INC:	(INC <n>)	→ <n+1>	
R-ADD:	((ADD <a>))	→ <a+b>	[模 2^32 回绕]
R-MUL:	((MUL <a>))	→ <a · b>	[模 2^32 回绕]
R-LT:	((LT <a>))	→ <1> 若 a<b(有符号) 否则 <0>	
R-DUP:	DUP	子树最多求值一次, 所有共享者看到同一结果 (惰性 + 记忆化)	

实现要点: 用带引用计数的图节点池, 不用树;R-S 的 x 必须共享, 否则 Church 数值上指数爆炸 (这是**功能错误**而非性能问题); 整数字面量即值; 必须设步数上限 (10^7), 超限报错。

16.3 强制测试对 (验收基准)

编号	字节序列	期望结果
T-A1	01 01 11 00 29 00 00 00 00 63 00 00 00	整数 41
T-A2	01 01 01 10 11 11 00 07 00 00 00	整数 7
T-A3	01 13 00 04 00 00 00	整数 5
T-A4	01 01 14 00 06 00 00 00 00 07 00 00 00	整数 13
T-A5	01 01 15 00 06 00 00 00 00 07 00 00 00	整数 42
T-A6	01 01 16 00 03 00 00 00 00 07 00 00 00	整数 1
T-A7	01 01 16 00 09 00 00 00 00 07 00 00 00	整数 0

T-A1~A3 同时写入 ASCII 头 (<len1..3> = 13, 11, 7)。

16.4 共享引用 (0xFF)——可选

打包器可对重复 DUP 子树发出 0xFF n(n 为本项内第 n 个 DUP 子树, 从 0 计); 解释器允许仅实现”见到 0xFF 即报错”的最小版本但须声明。v2 打包器默认关闭。

17. 第 2 层:QuipuVM v0.2 指令集

17.1 机器模型

状态:

R[0..15]

16 个 32 位补码寄存器

PC

字节地址, 始终为 8 的倍数

MEM

64 MiB 扁平字节数组, 初始全零 (允许稀疏实现)

RSTK

返回地址栈, 程序不可读写, 容量 ≥ 65536

无标志寄存器。比较结果写寄存器 (0 或 1)。

17.2 指令格式: 定长 8 字节

字节:

[0]

[1]

[2]

[3]

[4..7]

opcode

rd

ra

rb

imm32(小端)

每条指令 8 字节, 取指后默认 PC += 8; 不使用的字段**必须为 0**(打包器应校验); 跳转目标为**绝对字节地址**且必须是 8 的倍数, 否则 err=5。

17.3 入口与停机约定

- 启动:PC=0;R0 = in_len;R1 = 0(输入基址); 其余寄存器为 0; 输入已置于 MEM[0, in_len)。
- 正常停机 (HALT):R1 = 输出起始地址,R0 = 输出长度; 输出 = MEM[R1, R1+R0),err=0。R1+R0 越界 → err=2。
- 异常停机: 任何错误 → err = 错误码, 无输出。

17.4 指令表 (22 条)

opcode	助记符	字段	语义
0x00	HALT	—	正常停机 (见 17.3)
0x01	LI	rd, imm	R[rd] = imm32(原样, 补码)
0x02	MOV	rd, ra	R[rd] = R[ra]
0x08	ADD	rd, ra, rb	R[rd] = R[ra] + R[rb](环绕)
0x09	SUB	rd, ra, rb	R[rd] = R[ra] - R[rb](环绕)
0x0A	MUL	rd, ra, rb	R[rd] = R[ra] * R[rb](环绕)
0x0B	DIV	rd, ra, rb	有符号除, 向零取整;rb=0 → err=1;INT_MIN/-1 → INT_MIN
0x0C	MOD	rd, ra, rb	余数符号同被除数;rb=0 → err=1;INT_MIN%-1 → 0
0x0D	AND	rd, ra, rb	按位与
0x0E	OR	rd, ra, rb	按位或
0x0F	XOR	rd, ra, rb	按位异或
0x10	NOT	rd, ra	R[rd] = ~R[ra]
0x11	SHL	rd, ra, rb	左移, 计数 = rb & 31
0x12	SHR	rd, ra, rb	逻辑右移, 计数 = rb & 31
0x13	SAR	rd, ra, rb	算术右移, 计数 = rb & 31

Table 7 – continued

opcode	助记符	字段	语义
0x14	EQ	rd, ra, rb	$R[rd] = (R[ra] == R[rb])$ $? 1 : 0$
0x15	LT	rd, ra, rb	有符号小于 $\rightarrow 1/0$
0x16	LTU	rd, ra, rb	无符号小于 $\rightarrow 1/0$
0x18	LD8	rd, ra, imm	$R[rd] = MEM[R[ra]$ $+imm]$ (零扩展); 越界 $\rightarrow$ $err=2$
0x19	LD32	rd, ra, imm	4 字节小端加载; 越界 $\rightarrow$ $err=2$
0x1A	ST8	rd, ra, imm	$MEM[R[ra]+imm] = R[rd]$ $\& 0xFF$; 越界 $\rightarrow err=2$
0x1B	ST32	rd, ra, imm	4 字节小端存储; 越界 $\rightarrow$ $err=2$
0x1C	JMP	imm	$PC = imm$; 非 8 对齐 $\rightarrow$ $err=5$
0x1D	JZ	ra, imm	$R[ra] == 0$ 则 $PC = imm$ (对齐 检查同上)
0x1E	CALL	imm	RSTK 压入 $PC+8$; $PC = imm$; 栈满 $\rightarrow err=6$
0x1F	RET	—	RSTK 空 $\rightarrow err=4$; 否则 $PC =$ 弹出
0x3F	HALT_ERR	imm	异常停机, $err = imm \&$ $0xFF$ (不得为 0)

未列出的 opcode $\rightarrow err=3$ 。

17.5 错误码总表

码	含义
0	正常停机
1	除零
2	内存越界
3	非法 opcode
4	返回栈空时执行 RET
5	跳转目标非 8 对齐
6	返回栈溢出

17.6 边界行为钉死 (防歧义)

- 地址表达式 $R[ra] + imm$ 按**无符号 32 位回绕**计算后再判越界;
- 输入区与输出区允许重叠, 无特殊保护;
- LD8 永远零扩展 (符号扩展用 SHL 24 + SAR 24 合成);
- 单条指令内 $rd == ra == rb$ 合法, 语义为先读全部源再写目的。

18. QuipuVM 的 SKI 解释器 (权威语义)

作为 type=program codec=ski 节区存放。功能要求 (行为等价即可, 内部实现自由): 输入为一段 QuipuVM 字节码与一段输入字节 (编码为 Church 编码字节列表或等价表示, 须在节区头注释); 输出为二元组 (err, 输出字节列表)。必须逐条实现 17.4 的 22 条指令与 17.5 的错误码。

18.1 指令级测试向量 (26+ 条, 全部强制)

以 type=test 节区存放,ASCII 行格式:

```
TEST <id> ins=<8 字节指令 hex> code=< 程序 hex> in=< 输入 hex> expect_err=<d> expect_out=<hex>
```

必须覆盖:22 条指令各至少 1 条正常路径;err=1~6 各至少 1 条 (必须真实触发, 用 HALT_ERR 透传不算);DIV INT_MIN/-1、SHL 计数 33(验证 &31)、LD8 零扩展、rd==ra==rb 各 1 条。

19. 节区与清单

19.1 SEC 头

```
SEC id=<n> type=<t> codec=<c> len=<bytes> crc32=<decimal>\n
```

id 从 0 递增不重复;type spec | program | data | test | font;codec none | ski | quipuvvm | rle | dpcm | raw。解析器必须拒绝未知 type/codec(显式失败, 不静默跳过)。

19.2 编解码器语义

codec	负载含义	解码方式
none	已是最终形式 (ASCII 文本、测试向量)	直接读
raw	原始文件封装 (B/C 级)	不解码, 直接还原
ski	SKI 图项	SKI 求值器
quipuvvm	QuipuVM 字节码	QuipuVM 解释器
rle / dpcm	压缩媒体数据	第 20 章的随档解码程序

rle/dpcm 解码器必须以 type=program codec=quipuvvm 节区存在于同一档案中;MANIFEST 给出映射:CODEC rle = program section <id>。

19.3 MANIFEST 格式

```
MANIFEST
CODEC rle = program section 3
FILE id=5 name="docs/spec-jpeg.txt" type=spec codec=none len=81234 crc32=...
FILE id=6 name="img/disc.rle" type=data codec=rle len=404 crc32=...
TESTSUITE section=2 vectors=26
END-ARCHIVE
```

每节区一行 FILE;name 用双引号, 内部仅允许可打印 ASCII(0x20-0x7E, 不含 ")。MANIFEST 是冗余索引: 丢失后扫描全部 SEC 头可重建。

20. 标准编解码器规格 (随档程序)

20.1 RLE(图像/通用)

输入 (32 位字序列, 小端):word[0]= 宽 w,word[1]= 高 h,word[2]= 对数 p, 随后 p 对 (count, value)。输出 = 逐对展开的字节流; 总长度必须 = w*h, 否则程序必须 HALT_ERR 7。QuipuVM 参考实现 30 条指令。测试向量:49 对、32×32 圆盘, 输出首字节 0, 中央字节 255。

20.2 DPCM(音频)

输入:word[0]= 样本数 n,word[1]= 采样率,word[2]= 阶数 (v2 固定为 1),word[3]= 初值 s0, 随后 n 个有符号残差。输出 = n 个 16 位小端样本:s_i = clip16(s_{i-1} + r_i),clip 到 [-32768, 32767]。测试向量:64 样本正弦四分之一周期, 给出输出哈希与前 8 个样本真值。

20.3 新编解码器加入规则

必须同时提供:(a) QuipuVM 解码程序节区;(b) 3 条测试向量;(c) MANIFEST 的 CODEC 映射行。缺一项, 打包器必须拒绝。

21. 打包器 (pack) 与验证器 (verify)

21.1 pack

```
pack < 目录 > -o < 档案.quipu> [--level-map < 文件 >]
```

行为: 按 level-map 分级 (默认:.txt/.md → A 级,.jpg/.png/.mp4/... → B 级)→ 生成 ASCII 头 → 依次写入 SKI 解释器、测试向量、QuipuVM 编解码器、spec、数据节区 → 生成 MANIFEST 并回填全局 CRC → 自检: 立即执行一次完整 verify, 失败则删除输出并退出非零。性能: 吞吐 磁盘顺序读速的 80%; 除 A 级外不得执行任何解码/编码。

21.2 verify

```
verify < 档案.quipu>
```

按序执行, 全部 PASS 方可输出 ARCHIVE OK:

- 1. 结构:END 定位、SEC 合法性、负载长度吻合、MANIFEST 一致;
- 2. CRC: 每节区负载 CRC32 + 头中全局 CRC;
- 3. SKI 层:T-A1~A7 全部归约到期望值;
- 4. QuipuVM 层:test 节区全部向量通过;
- 5. 端到端: 用档案内 QuipuVM 编解码器解出每个 rle/dpcm 数据节区, 输出与已知哈希一致;
- 6. 独立性: 步骤 3-5 必须使用 verify 自带、按本规格第 16、17 章独立实现的解释器, 不得复用打包器内部代码。

21.3 里程碑

- M1:SKI 求值器 + T-A1~A7;
- M2:QuipuVM 解释器 (宿主语言版)+ 26 条指令向量;
- M3: 容器读写 + CRC + MANIFEST;
- M4:QuipuVM 的 SKI 解释器 + 双层端到端;
- M5:pack/verify 工具链 + 演示档案 (圆盘图 + 1 秒正弦音 + 本白皮书);
- M6:C 子集 → QuipuVM 编译器,inflate(固定哈夫曼子集) 移植;
- M7: 灯塔机构试点。

22. 常见实现陷阱 (检查单)

- 1. 小端: 所有 imm32、LD32/ST32、整数原子。先写” 存 0x01020304 后逐字节读出” 的测试;
- 2. DIV/MOD 向零取整, 先测 INT_MIN/-1、-7/2、7/-2 再写指令;
- 3. 移位计数必须 & 31,SHR 是逻辑移;
- 4. 指令未用字段必须为 0; 跳转地址必须 8 对齐;
- 5. SKI 的 R-S 必须共享 x(DUP), 朴素复制是功能错误;
- 6. HALT 的输出声明 (R1, R0) 同样查越界;
- 7. ASCII 行只允许 \n, 遇 \r 报错;
- 8. CRC32 一律 IEEE(zlib 默认), 不是 CRC32C;
- 9. 图项解码遇未知首字节必须报错;
- 10. 测试对是规范的一部分: 冲突时以测试对为准, 并回报文档缺陷。

附录: 最小完整档案骨架

```
[ASCII 头, 第 15 章模板, ~2400 字节]
END
SEC id=0 type=program codec=ski len=<n0> crc32=<c0>      ← QuipuVM 的 SKI 解释器
<n0 字节图项 >
SEC id=1 type=test codec=none len=<n1> crc32=<c1>        ← 26 条指令向量 (ASCII)
<n1 字节 >
SEC id=2 type=program codec=quipuvvm len=<n2> crc32=<c2>  ← RLE 解码器 (≤240 字节)
<n2 字节 >
SEC id=3 type=data codec=rle len=404 crc32=<c3>          ← 32×32 圆盘 RLE
<404 字节: 20 00 00 00 20 00 00 00 31 00 00 00 ... >
MANIFEST
CODEC rle = program section 2
FILE id=0 name="quipuvvm-interp.ski" type=program codec=ski ...
FILE id=1 name="quipuvvm-tests.txt" type=test codec=none ...
FILE id=2 name="rle-decoder.vvm" type=program codec=quipuvvm ...
FILE id=3 name="img/disc.rle" type=data codec=rle ...
TESTSUITE section=1 vectors=26
END-ARCHIVE
```

第四部分总结与号召

23. 局限与诚实的风险清单

- 1. 协调风险: 格式价值与使用人数正相关, 冷启动依赖灯塔机构; 历史上唯一反复奏效的办法是声望机构先押注;
- 2. 前提 2 的脆弱性: 若英文与 ASCII 双双消亡, 第 1 层失效——对此的兜底是时间分层 (第 3.2 节): 退到自然语言层, 再退到石刻图形规则卡;
- 3. AI 读者的能力边界: 我们假设未来 LLM 能读规格、写 60 行求值器、跑测试。当前模型已可做到, 但该假设随模型形态演变需要持续检验;
- 4. 不解决介质问题: 奇普是格式方案, 不是存储介质方案; 比特层面的存活依赖常规多副本冗余;
- 5. 知识产权: 随葬的规格文档与解码程序需妥善处置许可。

24. 与现有方案的关系

方案	哲学	差异
永续迁移（主流） PDF/A、WARC 等容器	跟着时代跑 规范当代格式	奇普：一次性付清，不再跑 奇普可嵌入其工作流，提供”执行能力” 这一它们没有的层
LOCKSS/冗余备份 区块链存证	多副本保命 信任活着的网络	解决介质层，与奇普互补 奇普：信任封存的测试对，无需任何活着的参与者
陶瓷/蚀刻介质	介质长寿	介质上仍写着会死的格式；奇普回答”刻什么”

25. 结语与号召

数字保存的深层困境，从来不是”比特能存多久”，而是”意义能活多久”。我们习惯把意义寄存在同时代的软件里，于是每十年为续命奔波一次。

四百年前的安第斯已经演示过结局：绳结还在，读绳的人不在了，于是一个文明被宣判”没有文字”。那把丢失的钥匙，成本不过一本手册。今天我们有 JPEG、基因序列、粒子对撞数据、整座数字图书馆——它们的钥匙寄存在 x86 和当代软件栈里，而这些东西的寿命，可能还不如印加的记录者。

奇普计划要做的事只有一件：趁钥匙还在我们手里，把它刻进数据自己。

- 给决策者：这不是一个公益幻想，而是一笔能算清的账——用不足 1% 的一次性存储开销，永久注销每十年一次的迁移预算。第一份试点档案的成本，低于一次常规迁移项目的零头。
- 给图书馆、档案馆、数据仓：你们的使命就是对抗断代。奇普不要求你们改换工作流——它作为 WARC/BagIt 里新增的一层，从今天起就可以打包第一批馆藏。
- 给工程师与 AI 实验室：第三部分就是全部规范。写一个 SKI 求值器只要 60 行，写一台 QuipuVM 只要一个下午——然后你就拥有了一份能被一万年后的读者打开的档案。M5 演示档案是最小的完整闭环，欢迎从那里开始。
- 给每一个写数据的人：下次导出文件时，多问一句——一百年后，谁还记得怎么读它？

绳结不该再等下一个四百年。让我们把钥匙系在绳上，现在就系。

本文档第一部分至第二部分为叙事与架构说明（信息性）；第三部分为规范性技术规格。两者冲突时，以第三部分与测试向量为准。